

Contents

Stateful Swarms for Document Intelligence: Auditability, Cost Efficiency, and Persistent Learning in Multi-Agent Systems	1
Abstract	1
1. Introduction	2
2. The Memory Problem in Agentic Systems	2
2.1 Context Windows Are Not Memory	2
2.2 Why RAG Does Not Solve This	3
2.3 What Structured Persistent State Looks Like	4
3. Architecture: The Blackboard Pattern	4
3.1 Core Design	4
3.2 Why the Blackboard Solves the Memory Problem	5
3.3 Model Routing	5
3.4 Signal Lifecycle	6
4. Evaluation	6
4.1 Benchmark	6
4.2 Results	6
4.3 Cost Analysis	7
4.4 What the Cost Structure Reveals	7
5. Auditability	7
5.1 The Case for Structured Reasoning Traces	7
5.2 Auditing Perfect Scores	8
5.3 Auditing Failures	9
5.4 Implications for Professional Adoption	9
6. The Case for Persistent Stateful Swarms	10
6.1 The Benchmark Constraint	10
6.2 The Real Cost of Starting From Zero	10
6.3 What Persistence Enables	10
6.4 Cost Projections	11
6.5 Learning Through State, Not Weights	11
6.6 Evaluation Isolation	11
7. Related Approaches	11
8. Limitations	12
9. Conclusion	12
References	13

Stateful Swarms for Document Intelligence: Auditability, Cost Efficiency, and Persistent Learning in Multi-Agent Systems

Devansh — Irys AI (Iqidis, Inc.) June 2026

Abstract

We present a swarm-based architecture for long-context document analysis that coordinates multiple AI agents through a shared, evolving blackboard. Unlike monolithic single-prompt approaches, the system

decomposes tasks into planning, extraction, analysis, and synthesis phases — each producing structured, source-grounded state that can be inspected, debugged, and extended. We evaluate on the full 1,251-task Harvey Legal Agent Benchmark (LAB), achieving 83.74% pooled criteria pass rate and 17.75% strict all-pass at \$1.30 per task. We argue that the primary value of swarm coordination is not raw performance but three properties that monolithic systems cannot provide: full auditability of the reasoning process, predictable cost scaling, and the foundation for persistent cross-session learning that reduces marginal cost over time. We open-source the complete system, all benchmark outputs, and five annotated reasoning traces under the MIT license.

Repository: <https://github.com/dl1683/ant-irys>

1. Introduction

The dominant approach to applying large language models to professional document work is the single-pass pipeline: ingest documents, construct a prompt, call a model, return the output. This approach has three structural problems that become severe as task complexity increases.

Opacity. A single model call produces an answer but no explanation of how that answer was derived. When the output is wrong — a missed clause, an incorrect calculation, a misattributed party — there is no intermediate state to inspect. The failure is visible only in the output, not in the reasoning that produced it. For professional work where accountability matters, this is not a minor limitation.

Cost unpredictability. Single-pass systems send the entire document context to the most capable (and most expensive) model for every query. A simple factual extraction and a complex multi-document comparison cost the same. There is no mechanism for routing simpler subtasks to cheaper models or for avoiding redundant processing of previously analyzed material.

Statelessness. Each invocation starts from zero. A system that analyzed a 200-page credit agreement yesterday has no memory of that analysis today. Every follow-up question, every new document added to the same matter, triggers a full re-processing cycle. The cost of the tenth question is identical to the cost of the first.

We propose that multi-agent swarm coordination with structured shared state addresses all three problems simultaneously. The key insight is that the intermediate state — the blackboard — is not just an implementation detail. It is the primary artifact of the system, more valuable than the final output it produces.

2. The Memory Problem in Agentic Systems

Before describing our architecture, it is worth examining why statelessness is not merely an inconvenience but a fundamental failure mode in current AI systems — one that affects every domain, not just document analysis.

2.1 Context Windows Are Not Memory

Modern language models have context windows ranging from 128K to over 1M tokens. It is tempting to treat these as working memory — load everything into context and let the model reason over it. In practice, this breaks in three distinct ways.

Attention degradation over length. Multiple studies and practitioner reports have demonstrated that model performance degrades on information positioned in the middle of long contexts. A fact stated on page 3 of a 200-page document receives different attention than the same fact stated on page 97. This is not a minor effect — it directly causes missed extractions, forgotten constraints, and inconsistent reasoning. The model has “seen” the information in the sense that it passed through the attention mechanism, but it has not reliably encoded it into its working representation.

Context compaction destroys information. When agentic systems — coding agents, research assistants, document analysts — run long sessions, they inevitably hit context limits. The standard solution is context compaction: summarizing earlier conversation history to free up token budget for new information. This is a lossy operation. Every compaction cycle discards details that seemed unimportant at summarization time but may become critical later. A coding agent that compacted away the specific error message from 30 minutes ago now cannot reference it when the same error recurs. A document analysis agent that summarized a list of 47 contract provisions into “various standard provisions” has lost the ability to answer questions about any specific provision.

The problem is structural: compaction algorithms cannot know which details will matter in the future. They optimize for what appears relevant *now*, which is precisely the wrong optimization for long-running analytical work where relevance shifts as understanding deepens.

Session boundaries erase everything. Even without compaction, every new session starts from an empty context. The analysis performed yesterday, the entities identified last week, the contradictions flagged across a series of documents — all of it vanishes. Agentic coding systems illustrate this vividly: an agent that spent 45 minutes understanding a codebase’s architecture, identifying the relevant modules, and building a mental model of the data flow starts from absolute zero when the user opens a new session. The next session begins with the same “let me read the files” exploration, discovering the same architecture, re-identifying the same modules. Every session pays the full cost of understanding.

This is not a limitation of any specific product. It is an architectural property of systems that use the context window as their sole memory substrate. The context window is a scratchpad, not a database. Treating it as persistent storage guarantees information loss.

2.2 Why RAG Does Not Solve This

Retrieval-augmented generation (RAG) addresses part of the problem by storing documents in a vector database and retrieving relevant chunks at query time. But RAG has its own structural limitations for professional work:

RAG retrieves text, not understanding. A RAG system can find the paragraph that mentions a SOFR floor, but it cannot retrieve the *analysis* that the SOFR floor in the draft credit agreement deviates from the commitment letter by 50 basis points. That analysis was never stored — it existed only in the model’s output during a previous session and was discarded.

No provenance chain. Retrieved chunks carry their source document but not the analytical context in which they were previously used. When an attorney asks “what did we conclude about the SOFR floor?”, a RAG system can retrieve the relevant clause but cannot reconstruct the reasoning that produced the conclusion.

No conflict awareness. RAG systems do not track which retrieved facts contradict each other, which have been superseded by newer documents, or which carry different confidence levels. They return text fragments ranked by embedding similarity, not by analytical reliability.

Chunking destroys structure. Document chunking — splitting documents into fixed-size segments for embedding — destroys the hierarchical structure that is essential for legal and financial analysis. A clause that spans two chunks is retrieved incompletely. A table that is split across chunks loses its row-column relationships. Cross-references between sections in different chunks are severed.

The fundamental issue is that RAG stores *source material* but not *analytical state*. For professional work, the analytical state — the entity graph, the obligation map, the deviation list, the risk assessment — is far more valuable than the raw text it was derived from.

2.3 What Structured Persistent State Looks Like

The alternative is to treat the intermediate analytical state as a first-class persistent artifact. Instead of storing documents and retrieving chunks, the system stores typed, provenance-tracked analytical entries that evolve over time:

- **Observations** with source document, section, evidence quote, and confidence
- **Analyses** that cross-reference multiple observations and carry explicit reasoning chains
- **Calculations** with arithmetic steps and source values
- **Gaps** identifying what is missing from the current understanding
- **Contradictions** between entries, with resolution status
- **Obligations** derived from analysis, linked to the entries that produced them
- **Signals** — open questions that the system has identified but not yet answered

Each entry carries provenance metadata: which worker created it, during which iteration, from which source, with what confidence, and which other entries it supports, contradicts, or supersedes. This metadata is not annotation — it is the mechanism by which the system maintains analytical integrity across sessions.

When a new session begins, the system does not start from zero. It loads existing state, identifies open gaps and unanswered signals, and proceeds from where it left off. When a new document arrives, the system reconciles it against existing entries, identifies contradictions, and updates its understanding incrementally. The cost of the second session is a fraction of the first because the expensive extraction work has already been done.

3. Architecture: The Blackboard Pattern

3.1 Core Design

The system coordinates multiple AI agents through a shared blackboard — a structured, append-only knowledge base that evolves over multiple iterations. Each agent reads from the blackboard, performs a specific type of cognitive work, and writes its findings back as typed entries with source provenance.

The pipeline proceeds through ordered phases:

1. **Seed Planning.** Before any document is read in detail, a planning agent scans document structure (headings, tables, estimated complexity) and produces: (a) key questions the investigation must answer, (b) extraction focus areas per document, (c) an analytical framework, (d) completeness criteria, and (e) a task-specific state map defining what object types and fields must be populated.
2. **Parallel Extraction.** Worker agents read source documents and write structured findings to the blackboard. Each finding carries a type (observation, analysis, calculation, strategy, gap, contradiction),

source provenance (document name, section, evidence quote), confidence, and epistemic classification (fact, inference, expert opinion, adversarial claim).

3. **Analytical Review.** A stronger model reviews accumulated state, identifies cross-document patterns, resolves contradictions, and produces higher-order analytical entries that reference the evidence entries they synthesize.
4. **Convergence Check.** A supervisor assesses whether the blackboard is complete enough to answer the task. If not, it dispatches targeted follow-up workers with specific questions derived from identified gaps.
5. **Obligation Extraction.** The system converts blackboard state into concrete deliverable obligations — specific items that must appear in the final output, each traced back to source entries.
6. **Curation.** The most relevant entries are selected and ordered for synthesis, enforcing minimum density thresholds to prevent thin outputs.
7. **Synthesis.** The final deliverable is produced from curated state. Large tasks use sectioned synthesis, drafting each section independently to avoid output truncation.
8. **Verification.** Deterministic and model-based checks confirm the output meets structural requirements. Missing obligations trigger targeted augmentation.

3.2 Why the Blackboard Solves the Memory Problem

The blackboard pattern addresses the three failure modes identified in Section 2:

No attention degradation. Each worker operates on a focused subset of the source material — a specific document section, a specific set of prior findings, a specific set of open signals. No single model call needs to attend to the entire document corpus. The blackboard distributes the attention burden across many focused calls rather than concentrating it in one long-context call.

No compaction loss. The blackboard is append-only structured storage, not a conversation history subject to summarization. Every observation, analysis, calculation, and gap is preserved with full provenance. Nothing is summarized away. When the system needs to revisit an earlier finding, it queries the blackboard by entry ID, type, tag, or source document — not by hoping the information survived a compaction pass.

No session boundary erasure. The blackboard is a JSON document on disk, not an in-memory conversation state. It persists across sessions by default. The 2,400 entries produced during a credit agreement analysis are available tomorrow, next week, and next month — ready to be queried, extended, and reconciled with new information.

3.3 Model Routing

Not every phase requires the same model capability. The system routes work to models based on the cognitive demands of each phase:

Phase	Model Tier	Rationale
Extraction	Lightweight	High-volume, factual, parallelizable
Planning, Analysis, Synthesis	Mid-tier	Requires reasoning, cross-referencing

Phase	Model Tier	Rationale
Convergence, Verification	Mid-tier	Quality judgment

This routing is the primary mechanism for cost control. Extraction — which constitutes the majority of model calls — uses the cheapest available model. Only phases requiring genuine reasoning use more capable (and expensive) models.

3.4 Signal Lifecycle

The blackboard maintains a signal queue: typed questions, read requests, and investigation prompts that guide worker behavior. Signals are generated during seed planning and dynamically during extraction as gaps are discovered. Each signal tracks its status (open, addressed, expired) and which entries address it. This creates a self-correcting feedback loop: the system generates questions, workers attempt to answer them, and the convergence check evaluates whether the answers are sufficient.

This is qualitatively different from a multi-turn conversation where the user must manually identify what’s missing and ask follow-up questions. The signal lifecycle is self-directed — the system identifies its own knowledge gaps and dispatches work to fill them without human intervention.

4. Evaluation

4.1 Benchmark

We evaluate on the Harvey Legal Agent Benchmark (LAB), a public benchmark consisting of 1,251 tasks across 24 legal practice areas. Each task provides source documents (contracts, filings, correspondence, spreadsheets) and an instruction requiring the system to produce one or more deliverables (memos, analyses, redlines, spreadsheets). Tasks are scored against per-criterion rubrics; strict all-pass requires every criterion to be satisfied.

Harvey’s published results on their private holdout set (which mirrors the public distribution) report 10.4% strict all-pass at approximately \$50.90 per task. We do not have access to the private holdout and would welcome the opportunity to run on it for a direct comparison.

4.2 Results

Metric	Result
Tasks completed	1,251 / 1,251
Criteria pass rate	62,800 / 74,990 = 83.74%
Strict all-pass	222 / 1,251 = 17.75%
Success gate (≤ 2 misses or $\geq 95\%$)	336 / 1,251 = 26.86%
Total cost	\$1,626.08
Cost per task	\$1.30

Performance varies significantly by task type:

Task Type	Tasks	Criteria %
draft	427	90.22
analyze	89	86.42
scenario	119	83.91
review	58	80.86
compare	148	77.14
extract	138	75.67
identify	195	74.26

The system is strongest on generative tasks (drafting, analysis) where partial state can still produce useful output, and weakest on exhaustive recall tasks (extraction, identification) that require complete enumeration of specific facts. This asymmetry is informative: it reveals that the primary bottleneck is state completeness, not synthesis quality.

4.3 Cost Analysis

The \$1.30 per task average represents a 39x reduction compared to Harvey’s published \$50.90 figure. This reduction comes from three sources:

1. **Model routing.** The majority of extraction work uses lightweight models at \$0.25/M input tokens, while only planning, analysis, and synthesis use mid-tier models at \$1.50/M input tokens.
2. **Parallel extraction.** Workers process document sections concurrently, reducing wall-clock time without increasing per-token costs.
3. **Targeted follow-up.** Rather than re-processing entire documents when information is missing, the convergence check dispatches focused workers to specific sections, avoiding redundant processing.

4.4 What the Cost Structure Reveals

The cost breakdown by model tier is instructive. Approximately 70% of total token spend goes to lightweight extraction workers. These workers perform the bulk of document reading — the labor-intensive work of converting raw text into structured findings. The remaining 30% goes to mid-tier models for planning, analysis, convergence checking, and synthesis — the “thinking” work that requires genuine reasoning.

This distribution has a direct implication for persistent state: if the extraction cost dominates, and extraction results can be cached, then subsequent queries against the same document set should cost roughly 30% of the initial analysis. The expensive work — reading and structuring the documents — has already been done.

5. Auditability

5.1 The Case for Structured Reasoning Traces

The most significant property of the blackboard architecture is not performance but auditability. Every task produces a complete, structured record of how the system arrived at its conclusions.

Each blackboard snapshot is a JSON document containing every entry the system has produced, with full provenance:

```

{
  "id": "e716",
  "type": "observation",
  "content": "Northbrook Capital Markets, LLC commits to provide a first lien senior secured te
  "source": {
    "document": "commitment-letter.docx",
    "section": "Full Document (part 1)"
  },
  "created_by": {
    "worker_id": "reader_commitment-letter.do",
    "description": "initial_reading",
    "iteration": 0
  },
  "confidence": 0.9
}

```

This entry records not just *what* was found, but *where* it was found, *who* found it (which worker), *when* in the process it was found (iteration 0), and *how confident* the system is in the finding.

Compare this to a system that outputs “The Term Loan B is \$350 million.” That statement may be correct, but there is no way to verify: Which document was it extracted from? Which section? Is there conflicting information elsewhere? How confident is the system? The blackboard entry answers all of these questions, and the answers persist as structured data that can be queried programmatically.

5.2 Auditing Perfect Scores

When the system succeeds, the reasoning trace demonstrates *why* it succeeded. In a credit agreement comparison task that scored 40/40, the blackboard evolved from 7 seed planning entries to 2,400 grounded findings over 12 iterations. The system:

- Generated 12 targeted questions before reading any documents
- Extracted 2,044 source-grounded observations with exact dollar amounts, dates, and clause references
- Produced 87 calculations cross-referencing terms across documents
- Identified 135 gaps requiring follow-up extraction
- Built 113 cross-document analysis entries identifying specific deviations

The final analysis entries contain findings such as:

```

{
  "id": "e865",
  "type": "analysis",
  "content": "Section 2.06 of the draft credit agreement specifies an annual agency fee of $50,
  "created_by": {
    "worker_id": "flash35_analyst",
    "description": "direct_analysis",
    "iteration": 12
  },
  "confidence": 0.98,
  "supports": ["e260", "e261", "e35"]
}

```


}

The supports field creates an explicit evidence chain: this analysis entry is grounded in three earlier observations, each of which carries its own source provenance. A reviewer can follow the chain from conclusion to evidence to source document.

5.3 Auditing Failures

More importantly, when the system fails, the reasoning trace reveals *why* it failed — and whether the failure is fundamental or fixable.

In a sanctions entity extraction task (80/85), the system missed five criteria. Examining the blackboard reveals that in every case, the relevant information was present in the state but the final verification step was incomplete:

Missed criterion: “Beneficiary name inconsistency.” The blackboard contained two entries with different name variants — “Zenith Petrochem Industries LLC” in one entry and “Zenith Petrochemical Industries LLC” in another. Both name variants existed in the state. The system extracted the facts correctly from both documents. What it failed to do was cross-reference these entries and flag the discrepancy. The information was there; the cross-reference step wasn’t.

Missed criterion: “OFAC 50% rule aggregation principle.” The system identified the threshold proximity (49% vs 50%) and even mentioned that aggregate ownership by blocked persons could trigger a violation — but didn’t elaborate on the aggregation principle with sufficient specificity for the scorer.

This pattern — correct extraction, incomplete verification — is qualitatively different from a system that never found the relevant information. It indicates that the failure mode is in the state processing pipeline, not in the extraction capability. These are fixable failures, and the blackboard makes it possible to identify them precisely.

A monolithic system that missed these same criteria would provide no insight into *why*. Did it not read the relevant document? Did it read it but not extract the entity? Did it extract the entity but not notice the name variant? With a blackboard, each of these failure modes produces a distinct signature that can be diagnosed and addressed.

5.4 Implications for Professional Adoption

For legal work specifically, auditability is not optional. Attorneys are professionally responsible for the accuracy of their work product. A system that produces correct answers 83% of the time is useful only if the remaining 17% can be identified and corrected. The blackboard pattern provides the mechanism for this: every conclusion is traceable to evidence, every evidence entry is traceable to a source document, and every gap is explicitly logged.

This has direct implications for professional liability. An attorney who relies on an opaque AI system that produces an incorrect conclusion has limited ability to explain *how* the error occurred. An attorney who relies on a system with a full reasoning trace can point to the specific entries, identify where the chain broke, and demonstrate that the error was a specific, bounded failure — not a systemic inability to reason.

Beyond legal work, this auditability property applies to any domain where decisions must be defensible: financial analysis, regulatory compliance, medical research synthesis, insurance underwriting, and due diligence across industries.

6. The Case for Persistent Stateful Swarms

6.1 The Benchmark Constraint

The results presented above were produced under a significant constraint: every task starts from an empty blackboard. The system has no memory of previous tasks, no accumulated domain knowledge, and no persistent document indexes. This is the correct approach for benchmark evaluation — persistent state would constitute an unfair advantage, as the system would be learning from the benchmark itself.

However, this constraint means the benchmark results systematically understate the performance and cost efficiency of a deployed system.

6.2 The Real Cost of Starting From Zero

Consider what happens when the system processes a credit agreement comparison. It spends the first several iterations performing extraction — reading documents, identifying parties, extracting financial terms, cataloging clauses. This extraction work produces roughly 2,000 observation entries at a cost dominated by lightweight model calls.

Now consider what happens when the same user asks a follow-up question about the same documents: “What are the conditions precedent?” The system must re-read the same documents, re-identify the same parties, re-extract the same financial terms, and re-catalog the same clauses — only to then focus on the specific question about conditions precedent. The extraction work from the first query is completely wasted.

In a real legal practice, matters persist for weeks or months. An M&A transaction involves dozens of document reviews against the same set of agreements. A litigation matter involves repeated analysis of the same filings as new evidence emerges. A compliance program involves ongoing monitoring against the same regulatory framework.

For a stateless system, every interaction with these ongoing matters pays the full extraction cost. For a stateful system, the extraction cost is paid once and amortized across every subsequent interaction.

6.3 What Persistence Enables

A persistent swarm would retain its blackboard state across sessions. The practical implications are:

Incremental document processing. When a new document arrives on an existing matter, the system reconciles it against the existing entity graph and obligation map rather than building those structures from scratch. A new amendment to a credit agreement triggers comparison against the already-extracted terms of the original agreement — the system identifies what changed without re-reading the original.

Accumulated entity knowledge. Named entities, their roles, their relationships, and their prior appearances across documents accumulate in the persistent state. The fifth time the system encounters “Northbrook Capital Markets” it already knows this is the administrative agent with specific fee obligations — it does not need to re-discover this from the source documents.

Gap-aware resumption. Open signals — questions the system identified but could not fully answer — persist across sessions. When new information becomes available (a new document, a clarification from the user), the system can immediately direct extraction toward the previously identified gaps rather than starting the investigation from scratch.

Background maintenance. A persistent system can perform background reconciliation: checking whether new documents contradict existing state, updating entity relationships as new information arrives, and

flagging stale entries that may need re-extraction.

6.4 Cost Projections

The cost structure of the current system is dominated by extraction. With persistent state, subsequent queries on the same document set would skip extraction entirely and proceed directly to analysis and synthesis from cached state.

Conservative estimates based on the observed cost breakdown suggest that for matters with 5+ queries against the same document set, persistent state would reduce marginal cost by 60-80% per additional query. The first analysis would cost roughly \$1.30. The fifth analysis of the same documents would cost a fraction of that — the extraction has already been done.

This changes the economic model of AI-assisted document analysis from “pay full price for every question” to “invest in understanding a document set once, then query cheaply forever.” For practices that work on recurring document types (credit agreements, compliance filings, standard contracts), the amortized cost per insight drops dramatically over time.

6.5 Learning Through State, Not Weights

It is important to distinguish persistent state from fine-tuning. The model weights remain unchanged. The system becomes more effective over time not because it has been retrained but because it has accumulated more structured context. This preserves the auditability property: every piece of accumulated knowledge is traceable to the specific document and extraction iteration that produced it.

This also preserves model flexibility. When a better model becomes available, the system can swap it in without retraining — the persistent state is model-independent. The blackboard entries are structured data, not model-specific representations. A blackboard produced by one model can be queried, analyzed, and extended by a different model.

6.6 Evaluation Isolation

Persistent state creates a tension with benchmark evaluation. If the system retains knowledge from previous benchmark tasks, its performance on subsequent tasks is no longer a clean measurement of its capability. This is why we deliberately strip all persistence for benchmark runs.

The solution is evaluation-aware persistence: strict boundaries between benchmark state and production state, separate storage contexts for evaluation and deployment, and audit mechanisms that verify no benchmark-derived knowledge contaminates generation. We have implemented a provenance audit system that monitors every model prompt for forbidden benchmark metadata (scoring criteria, expected answers, prior scores) and logs violations.

7. Related Approaches

The swarm architecture intersects with several active research directions that we believe are complementary:

Latent space reasoning. Inference-time perturbation of model activations can improve reasoning without training. We have demonstrated +19.6pp arithmetic improvement on frozen models through soft token

injection. Applied to swarm worker models, this could improve extraction accuracy on the precise numerical and citation tasks where the current system is weakest. See: <https://github.com/dl1683/Latent-Space-Reasoning>

Hierarchical embeddings. Standard dense embeddings treat all dimensions equally, but document semantics are hierarchical — a clause lives inside a section inside a document. Multi-scale embeddings that align dimensional structure to semantic hierarchy improve retrieval quality for document analysis. We have demonstrated that correct geometric hierarchy causally improves embedding quality. See: <https://github.com/dl1683/moonshot-fractal-embeddings>

Representation quality theory. We have derived a universal law governing learned representation quality from first principles using extreme value theory, validated across 12 NLP architectures ($R^2=0.955$) and biological neural systems. This provides a theoretical framework for predicting which models will produce the best representations for which task types — relevant for optimal model routing in multi-model swarms. See: <https://github.com/dl1683/moonshot-cti-universal-law>

Persistent agent memory. Provenance-backed knowledge memory systems that maintain source attribution, conflict state, and temporal validity across sessions are a necessary complement to the swarm pattern for production deployment. See: <https://github.com/dl1683/MapU>

8. Limitations

Recall on exhaustive tasks. The system achieves 90%+ on generative tasks but 74-77% on extraction and identification tasks requiring complete enumeration. The blackboard shows this is primarily a state completeness problem — the system doesn't always extract every relevant item from long documents.

Single-run benchmark. The results represent a single full benchmark run. We have not performed statistical analysis of run-to-run variance.

Public benchmark only. We do not have access to Harvey's private holdout set. The public and private sets are described as having the same distribution, but a direct comparison on identical evaluation data is not possible. We would welcome the opportunity to run on the private holdout.

No persistent state evaluation. The cost and accuracy benefits of persistent state described in Section 6 are projections based on the observed cost structure, not measured results from a deployed persistent system.

API-only. The system uses only standard API calls to frontier language models. It does not employ fine-tuning, custom embeddings, retrieval-augmented generation, or any model modifications. This is both a limitation and a deliberate design choice — it establishes a baseline for what coordination alone can achieve.

9. Conclusion

We have presented a swarm-based document analysis architecture that achieves competitive benchmark performance at dramatically lower cost than published alternatives. The system's primary contribution is not raw performance but three structural properties: full auditability of the reasoning process through structured blackboard state, predictable cost through model routing and targeted processing, and a natural foundation for persistent cross-session learning.

The memory problem in current agentic systems — context compaction destroying information, session boundaries erasing understanding, RAG retrieving text but not analysis — is not a minor inconvenience. It is the reason that AI-assisted professional work remains fragile and expensive. Systems that forget what they learned yesterday will always pay the full cost of understanding today. Structured persistent state offers a path out of this cycle: pay the cost of understanding once, then build on that understanding incrementally.

The complete system is open-source under the MIT license — use it however you want, including commercially. The repository includes source code, the full benchmark manifest, five annotated example tasks with complete reasoning traces, and verification instructions. Benchmark outputs for all 1,251 tasks are available for independent scoring.

We believe that stateful multi-agent coordination represents an important direction for professional AI systems. We invite collaborators interested in extending this work to additional benchmarks, new problem domains, or production deployment.

Contact: devansh@iqidis.ai **Repository:** <https://github.com/dl1683/ant-irys> **License:** MIT — use however you want, including commercially.

References

1. Harvey AI. “Legal Agent Benchmark Initial Results.” harvey.ai/blog/legal-agent-benchmark-initial-results, 2025.
2. Harvey AI. “Opus 4.8 Now Live in Harvey.” harvey.ai/blog/opus-4-8-now-live-in-harvey, 2026.
3. Harvey AI. Harvey Legal Agent Benchmark. github.com/harveyai/harvey-labs, 2025.